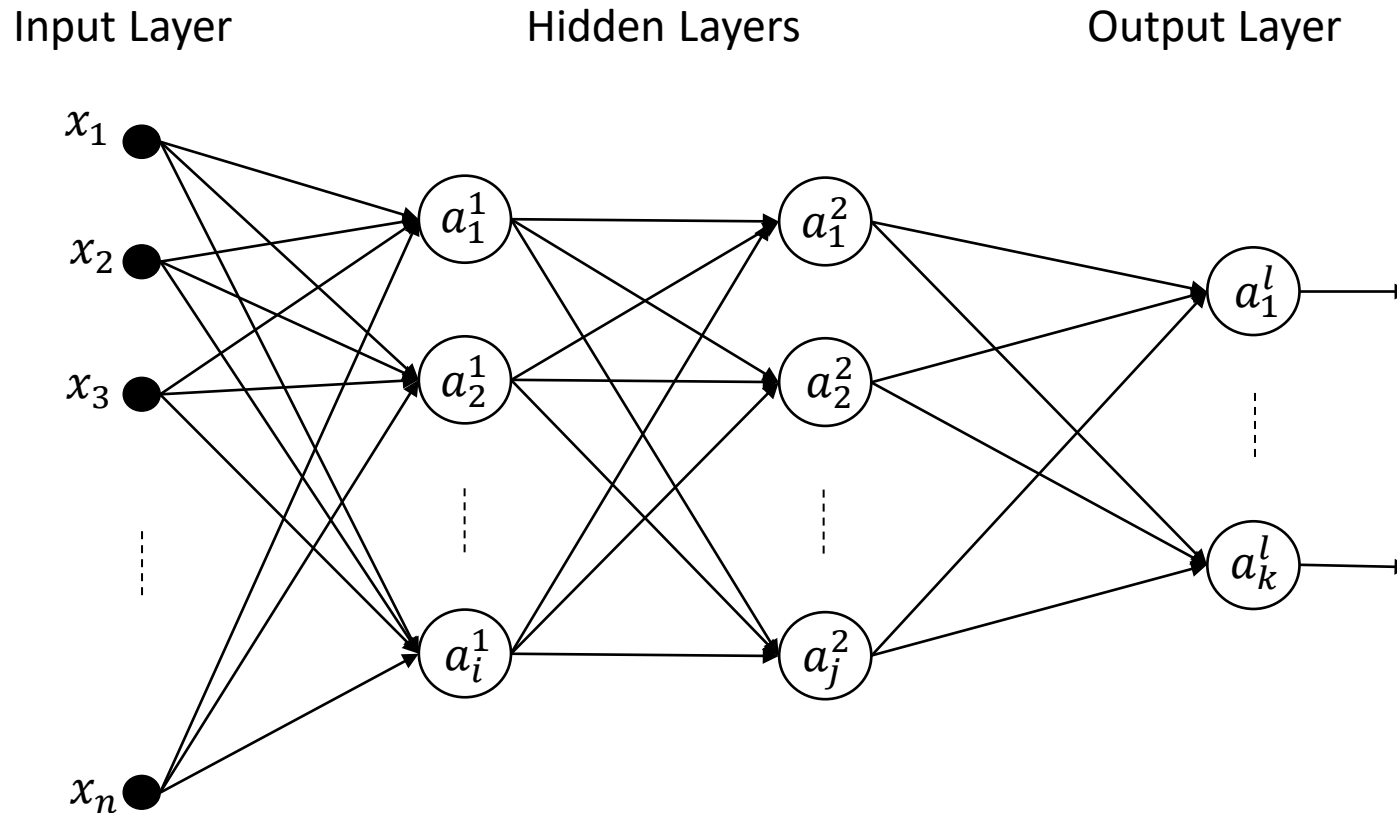


Lab2: ANNs Basic Maths

Using tensor operations on Python

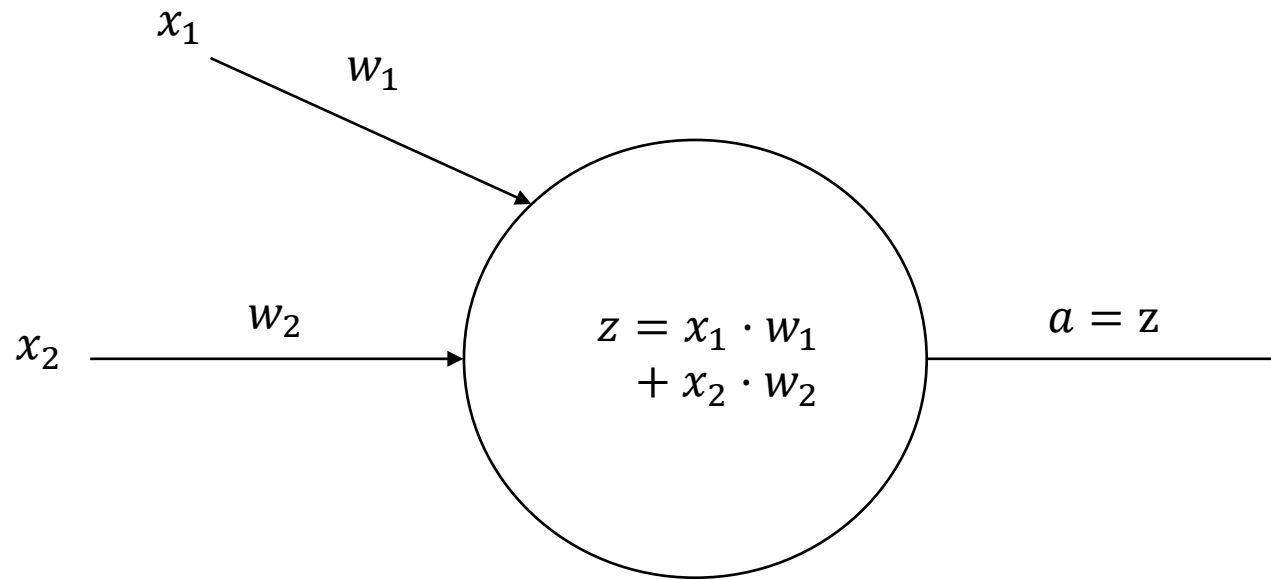
Artificial Neural Network Anatomy

- Example: fully connected, feed-forward simple network



Artificial Neuron

- Forward Propagation. Simple neuron (I)

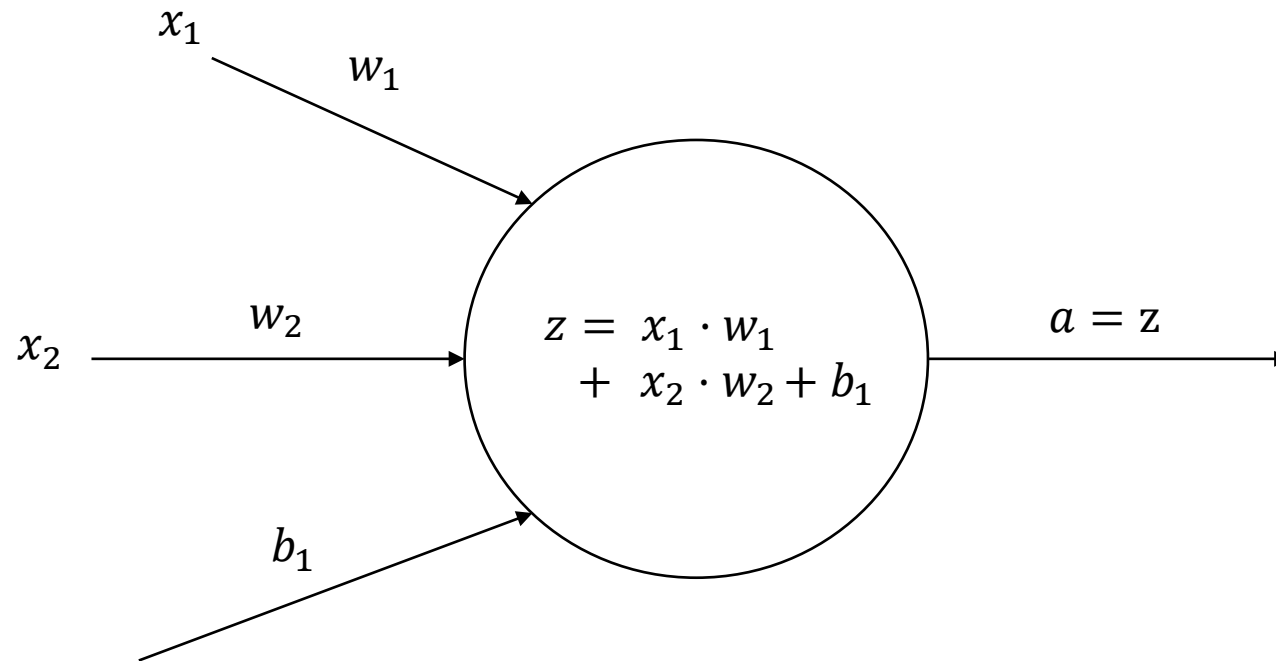


Z is just a weighted sum of the input.

ANN implements just a linear regression model.

Artificial Neuron

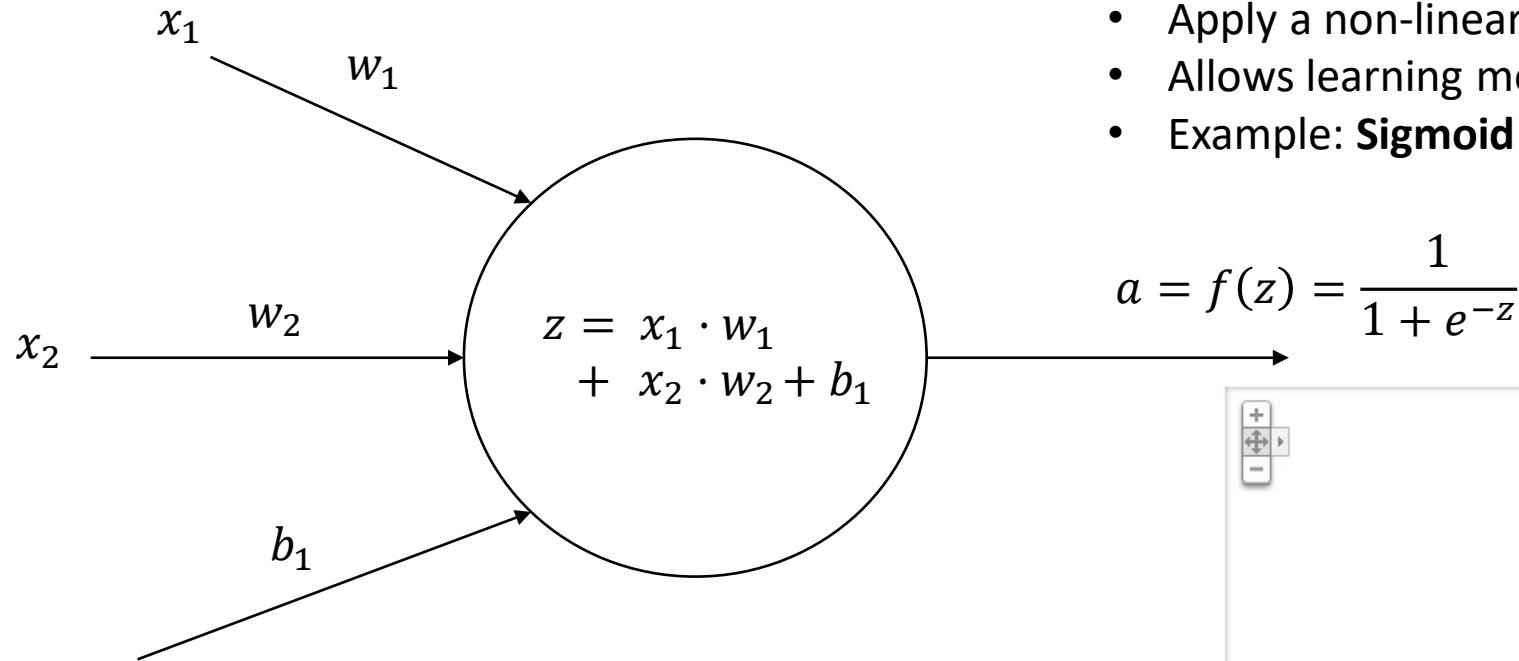
- Forward Propagation. Simple neuron (II)



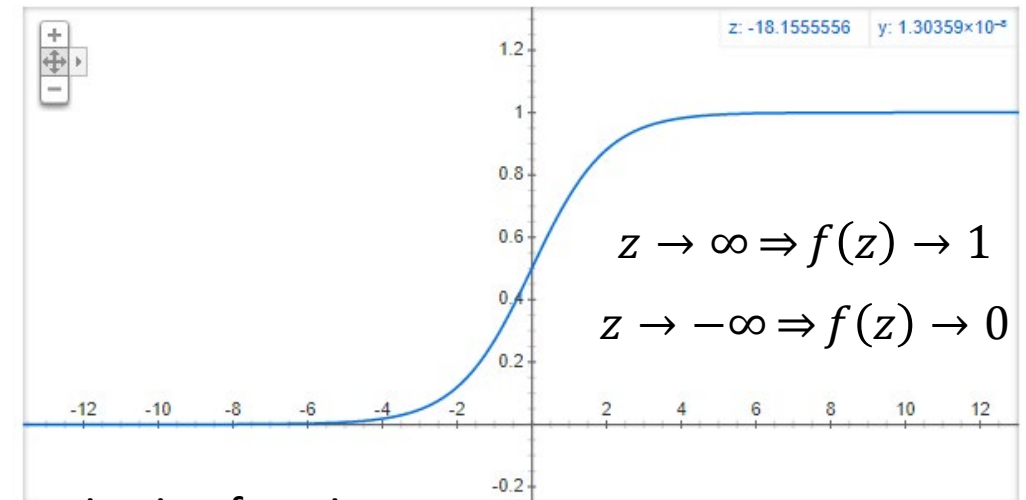
Adding an independent term, b , called the bias.

Artificial Neuron

- Forward Propagation. Simple neuron (III):



- Apply a non-linear transformation: activation function.
- Allows learning more complex features.
- Example: **Sigmoid activation function:**



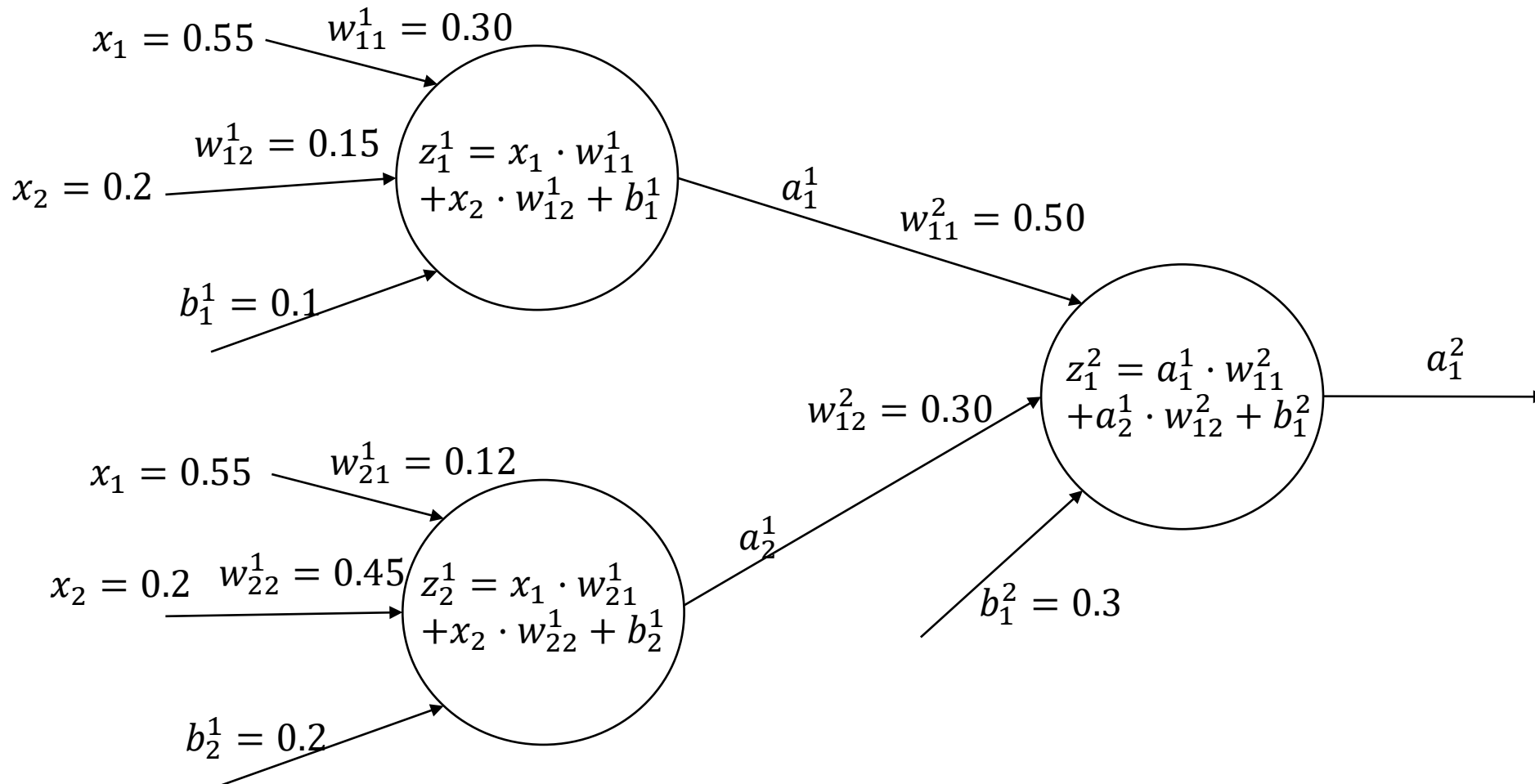
NOTE: Sigmoid has been substituted in most cases by better activation functions

Artificial Neural Network (ANN)

Numpy basic
maths for
NN.ipynb
(Part I)



- Forward Propagation. Compute output for the ANN:



Supervised Learning

- **Supervised Learning** is a method for finding the values of a network's parameters $(w_{i,j}^l, b_j^l)$ that make the model's output closest to the "real" output.
- The **network is trained** with input-output data pairs.
 - For a given input, the net output is calculated with the current parameters values.
 - The error between the net output and the "real" (desired) output for this input is calculated.
 - The network parameters are modified in a way that the error gets reduced.

Cost function

- CF is used to evaluate the degree a function fits some data (m samples).

$$J(w_j) = \frac{1}{m} \sum_{i=1}^m L(\hat{a}(i), a(i))$$

- A common **loss function** is the Mean Squared Error (MSE) between the Ground Truth and the Predicted Values, for each Input Data.

$$J(w_j) = \frac{1}{m} \sum_{i=1}^m (\hat{a}(i) - a(i))^2$$

NOTE: MSE has been substituted in most cases by better cost functions

- CFs are used in **regression analysis** (i.e. linear regression) to obtain the values of the model's parameters that make it best fit the data (<=>minimize the CF<=>minimize the error).

Gradient Descent

- **Gradient Descent** is an iterative method for finding the minimum of a given Cost Function.
- GD is used in regression analysis to find the values of a model's parameters that minimize the model error (CF).
- At each iteration, the parameters of the cost function are updated in the opposite direction of the Cost's gradient for this parameter:
$$w_j^{(i+1)} = w_j^{(i)} - \alpha \frac{\partial J}{\partial w_j}$$
$$J(w_j) = \frac{1}{m} \sum_{i=1}^m (\hat{a}(i) - a(i))^2$$
$$a = f(w_j)$$
- The **Learning Rate**, α , is a constant with an empirically determined value (0.1-0.0001, for ex.)

Gradient Descent example

- Simplest case: Linear regression

- Back propagation

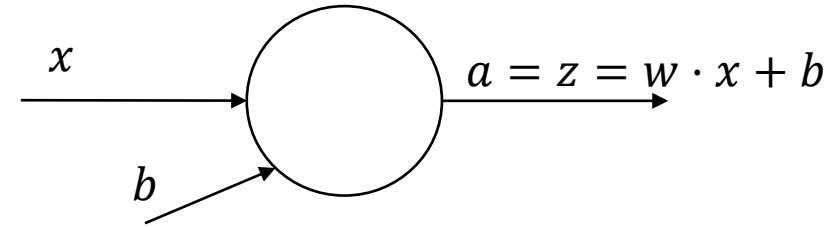
$$w^{(i+1)} = w^{(i)} - \alpha \frac{\partial J}{\partial w}$$

$$b^{(i+1)} = b^{(i)} - \alpha \frac{\partial J}{\partial b}$$

- Partial Derivatives

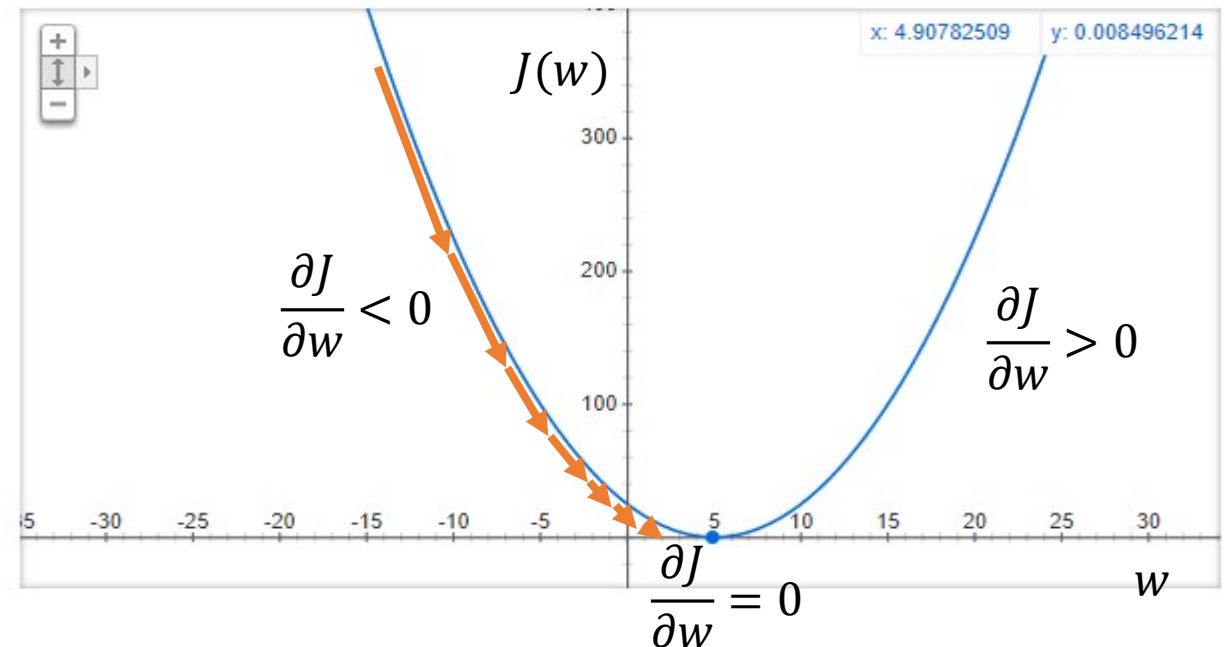
$$\frac{\partial J}{\partial w} = \frac{2}{m} \sum_{i=0}^m -x(i) [\hat{a}(i) - (w \cdot x(i) + b)]$$

$$\frac{\partial J}{\partial b} = \frac{2}{m} \sum_{i=0}^m -[\hat{a}(i) - (w \cdot x(i) + b)]$$



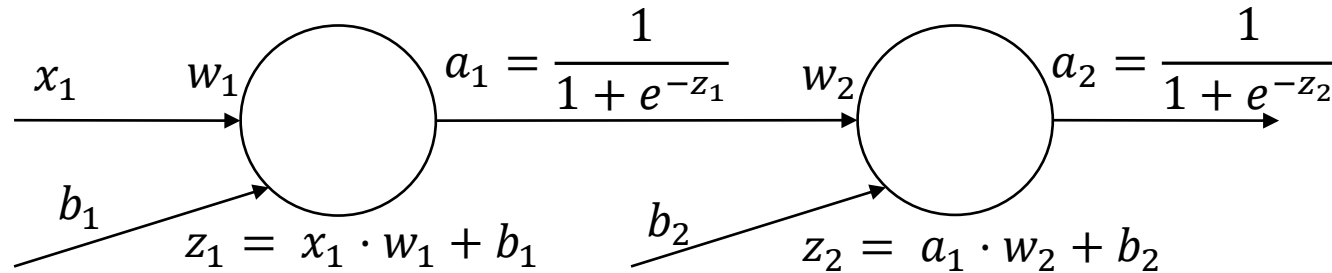
$$J(w, b) = \frac{1}{m} \sum_{i=1}^m [\hat{a}(i) - (w \cdot x(i) + b)]^2$$

– Why does it work? Visualization for a single parameter w ($b=0$):



ANN Backpropagation

- Simple ANN example:



$$J(w, b) = \frac{1}{2} (\hat{a} - a_2)^2$$

– For this particular network:

$$w_2^{(i+1)} = w_2^{(i)} - \alpha \frac{\partial J}{\partial w_2} \quad \frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2}$$

$$\frac{\partial J}{\partial w_2} = -(\hat{a} - a_2) \cdot a_2(1 - a_2) \cdot a_1$$

$$b_2^{(i+1)} = b_2^{(i)} - \alpha \frac{\partial J}{\partial b_2} \quad \frac{\partial J}{\partial b_2} = \frac{\partial J}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2}$$

$$\frac{\partial J}{\partial b_2} = -(\hat{a} - a_2) \cdot a_2(1 - a_2) \cdot 1$$

$$w_1^{(i+1)} = w_1^{(i)} - \alpha \frac{\partial J}{\partial w_1} \quad \frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

$$\frac{\partial J}{\partial w_1} = -(\hat{a} - a_2) \cdot a_2(1 - a_2) \cdot w_2 \cdot a_1(1 - a_1) \cdot x_1$$

$$b_1^{(i+1)} = b_1^{(i)} - \alpha \frac{\partial J}{\partial b_1} \quad \frac{\partial J}{\partial b_1} = \frac{\partial J}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1}$$

$$\frac{\partial J}{\partial b_1} = -(\hat{a} - a_2) \cdot a_2(1 - a_2) \cdot w_2 \cdot a_1(1 - a_1) \cdot 1$$

Training ANN by Gradient Descent

- Detailed algorithm:
 1. Initialize network parameters (w and b) with random values
 2. Iterate:
 1. Compute network output (forward propagation)
 2. Compute cost function (error between estimated and predicted output)
 3. Adjust parameters (w and b) by backpropagating the error
 4. Repeat until max iterations reached, or $\text{error} < \text{threshold}$

Batches and Epochs

- The **batch size** is a hyperparameter that defines the number of samples to work through before updating the internal model parameters. Popular batch sizes include 32, 64, and 128 samples.
 - **Batch Gradient Descent.** Batch Size = Size of Training Set
 - **Stochastic Gradient Descent.** Batch Size = 1
 - **Mini-Batch Gradient Descent.** $1 < \text{Batch Size} < \text{Size of Training Set}$
- The **number of epochs** is a hyperparameter that defines the number times that the learning algorithm will work through the entire training dataset. It can be set to 10, 100, 500, 1000, and even larger.

Vanishing Gradient Problem

- Sigmoid function gives values between 0 and 1. Thus, all intermediate output values in a network are between 0 and 1.
 - Moreover, $\frac{\partial a_i}{\partial z_i} \approx 0$ when z_i is just over +3 or under -3.
- In deep networks, backpropagation algorithm performs hundreds of multiplications involving factors (a_i) that are less than 1. Thus, the gradients tend to be smaller and smaller as we move towards the first layers.
- This implies that the weights of neurons in the firsts layers get updated by very small quantities, making learning very slow. In addition, accuracy becomes reduced as well.
- Conclusion: although convenient for the output layers (as it provides a normalized output), sigmoid needs to be replaced as an activation function in the intermediate layers.

Common Activation Functions

- Between others, the four most used activation functions are:
 - Sigmoid Function → already explained
 - Hyperbolic tangent (tanh)
 - ReLU (Rectified Linear Unit)
 - Softmax.

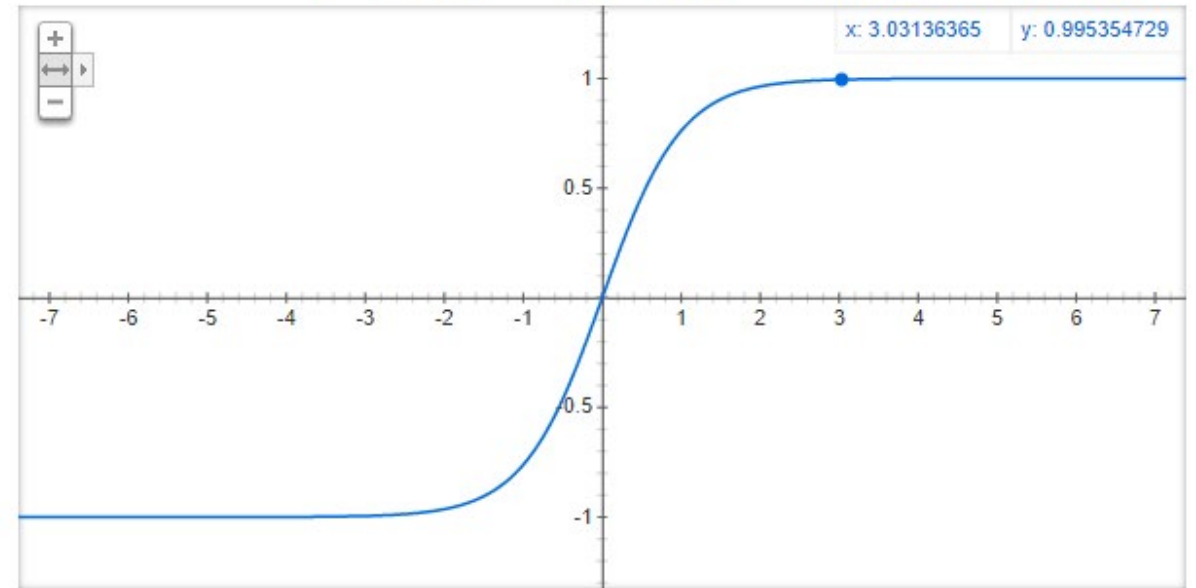
Common Activation Functions

- **Hyperbolic tangent (tanh)**

$$a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- Is a shifted version of the sigmoid function to make it symmetric, so that the output of the neurons could also be negative.
- It does not solves the vanishing gradient problem.

Gráfico de $(\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x))$



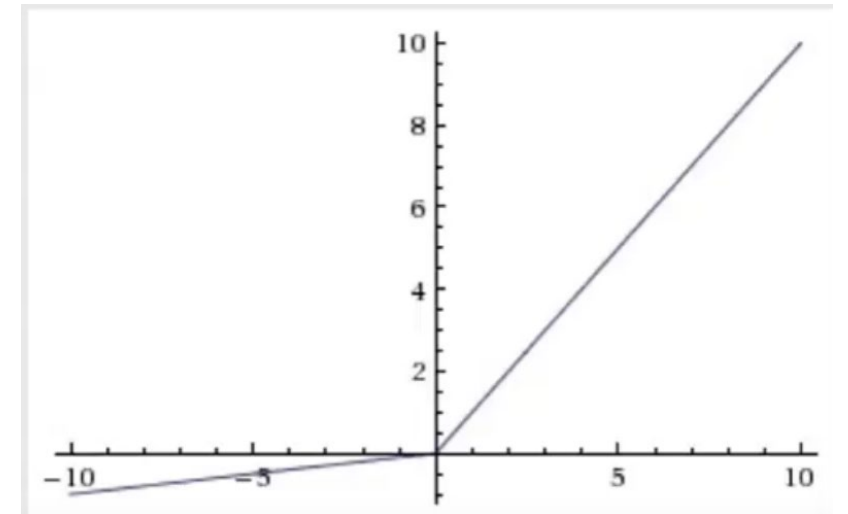
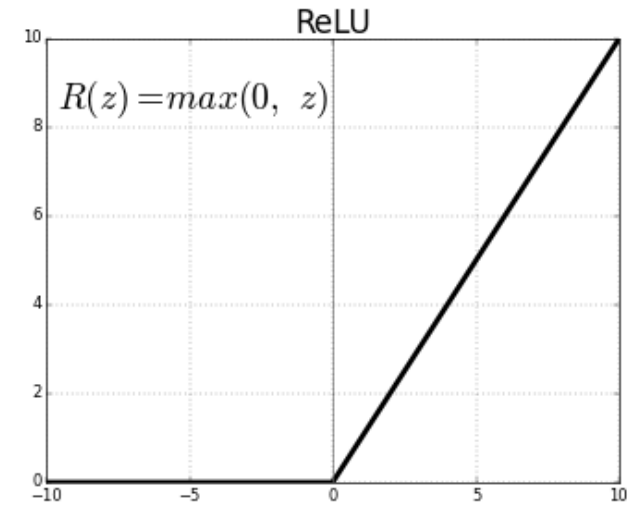
Common Activation Functions

- **Rectified Linear Unit (ReLU)**

$$a = \max(0, z)$$

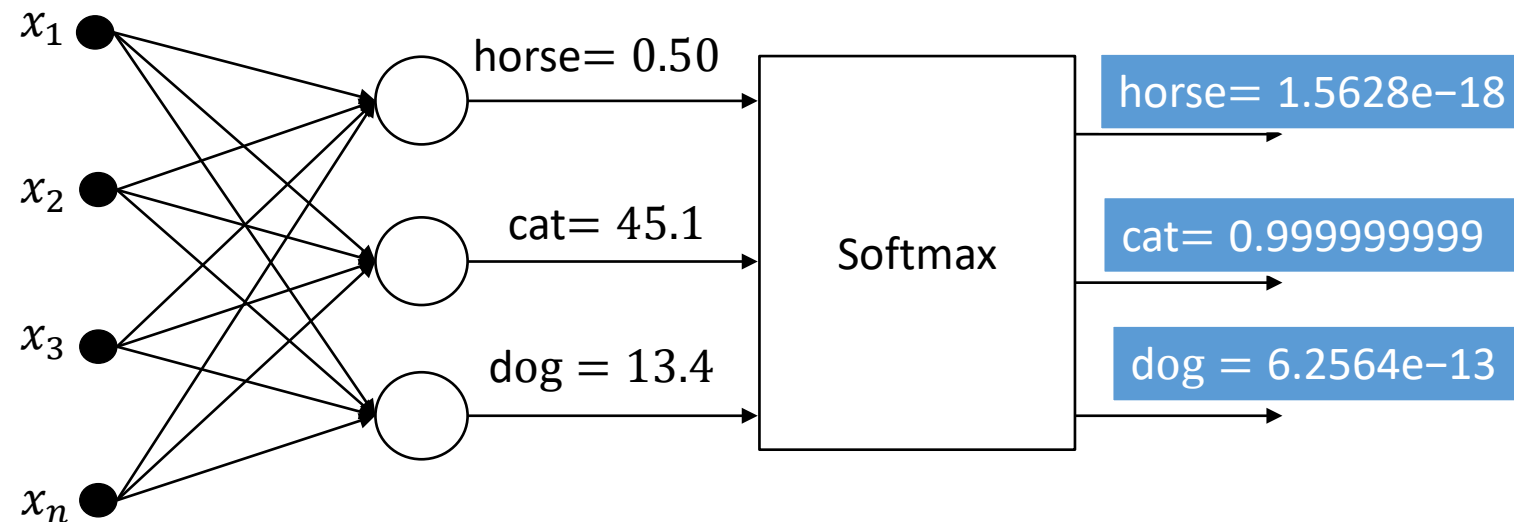
- Better gradient propagation.
Fewer vanishing gradient problems.
- Efficient computation.
- Scale-invariant:
 $\max(0, kz) = k \max(0, z)$
- Variant: Leaky ReLU

$$a = \begin{cases} 0.01z, & z < 0 \\ z, & z \geq 0 \end{cases}$$



Common Activation Functions

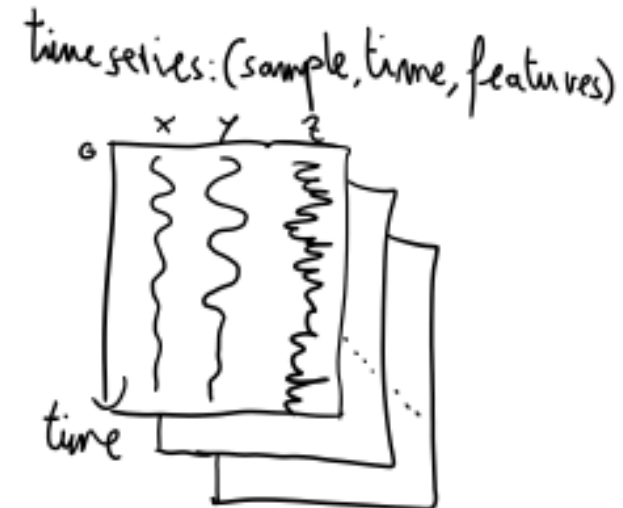
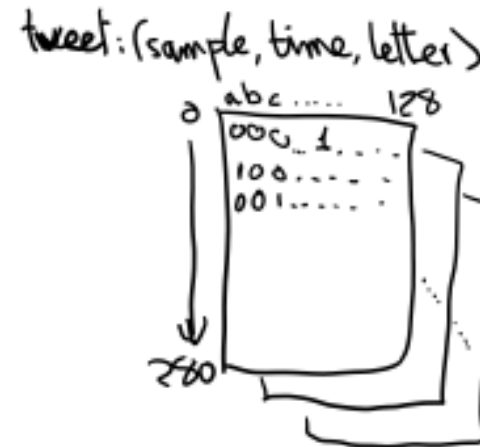
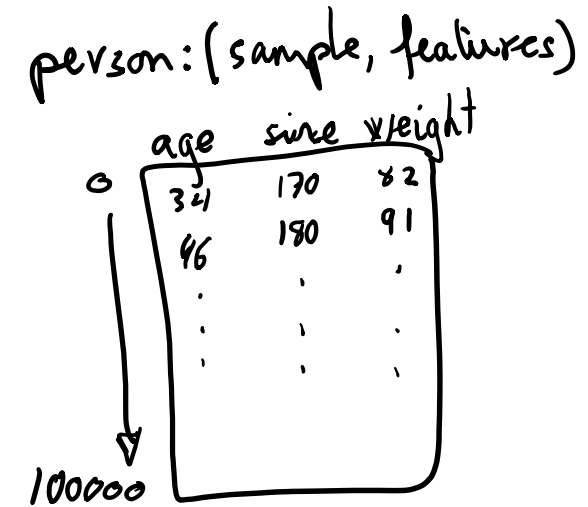
- **Softmax:** $\sigma(z_i) = \frac{e^{z_i}}{\sum_{k=1}^m e^{z_k}}$
 - It turns output values into probabilities, so that the sum of the probabilities is 1.
 - Useful for multiclassification models, as it returns the probabilities for all the output classes, having the target class the highest probability.



Tensor dimension ordering (I)

By convention, axis 0 is always the sample axis, time (if considered) is axis 1. Let's see some examples:

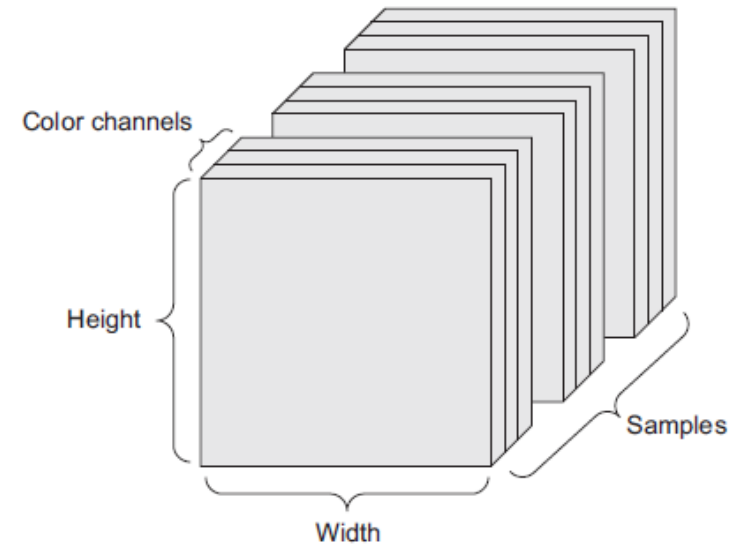
1. Vector data—2D tensors of shape (samples, features). A dataset of 100000 people including 3 features. age, size and weight: (100000,3)
2. Timeseries data or sequence data—3D tensors of shape (samples, timesteps, features). A dataset of 1000 tweets: 1000x280x128



Tensor dimension ordering (II)

By convention, axis 0 is always the sample axis, time (if considered) is axis 1. Let's see some examples:

3. Images—4D tensors of shape:
 - (samples, height, width, channels). This is tensorflow schema, or
 - (samples, channels, height, width). This is theano schema.
4. Video—5D tensors supports both formats. tensors of shape (samples, frames, height, width, channels) or (samples, frames, channels, height, width).



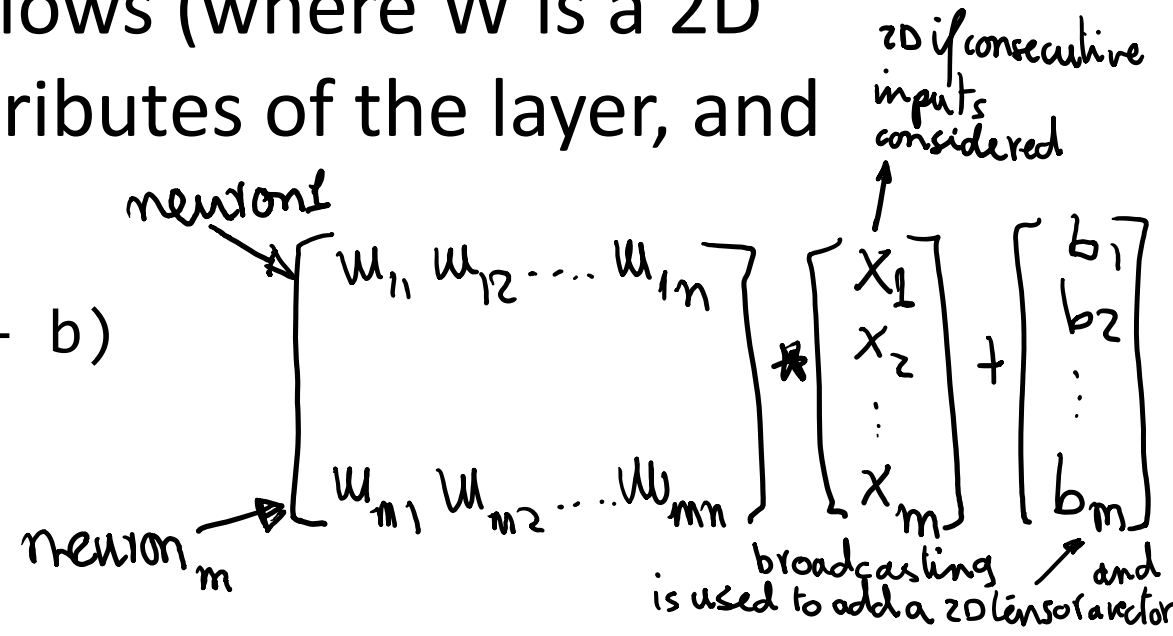
Tensor operations and keras

- A Keras layer instance looks like this:

```
keras.layers.Dense(512, activation='relu')
```

- This layer can be interpreted as a function, which takes as input a 2D tensor and returns another 2D tensor.
- Specifically, the function is as follows (where W is a 2D tensor and b is a vector, both attributes of the layer, and input is also a vector):

$$\text{output} = \text{relu}(\text{dot}(W, \text{input}) + b)$$



More... (as seen on the lecture slices)

- Hyperparameters:
 - Regularization
 - Metrics
 - Optimizers
 - ...

Lab3: Regression with keras

Doc on Jupyter notebooks

Lab4: Classification with keras

Doc on Jupyter notebooks

Deep Learning Best Practices

Gathering it all

Deep Learning workflow

1. Define the problem: What data is available, what are the input and output features to consider?
2. Identify a way to reliably measure success on your goal. For simple tasks, this may be prediction accuracy, but in many cases it will require sophisticated domain-specific metrics.
3. Define a training set, a validation set, and a test set. The validation-and test-set labels shouldn't leak into the training data
4. Vectorize the data by turning it into vectors and preprocessing it in a way that makes it more easily approachable by a neural network (normalization, and so on).
5. Develop a first model that beats a trivial common-sense baseline, thus demonstrating that machine learning can work on your problem. This may not always be the case!
6. Gradually refine your model architecture by tuning hyperparameters and adding regularization. Make changes based on performance on the validation data only, not the test data or the training data. Remember that you should get your model to overfit (thus identifying a model capacity level that's greater than you need) and only then begin to add regularization or downsize your model.

Selecting an architecture: dense networks (I)

- To perform *binary classification*, end your stack of layers with a Dense layer with a single unit and a sigmoid activation, and use `binary_crossentropy` as the loss. Your targets should be either 0 or 1:

```
from keras import models
from keras import layers
model = models.Sequential()
model.add(layers.Dense(32, activation='relu', input_shape=(num_input_features,)))
model.add(layers.Dense(32, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
model.compile(optimizer='rmsprop', loss='binary_crossentropy')
```

- To perform *single-label categorical classification* (where each sample has exactly one class, no more), end your stack of layers with a Dense layer with a number of units equal to the number of classes, and a softmax activation. If your targets are one-hot encoded, use `categorical_crossentropy` as the loss; if they're integers, use `sparse_categorical_crossentropy`:

```
model = models.Sequential()
model.add(layers.Dense(32, activation='relu', input_shape=(num_input_features,)))
model.add(layers.Dense(32, activation='relu'))
model.add(layers.Dense(num_classes, activation='softmax'))
model.compile(optimizer='rmsprop', loss='categorical_crossentropy')
```

Selecting an architecture: dense networks (II)

- To perform *multilabel categorical classification* (where each sample can have several classes), end your stack of layers with a Dense layer with a number of units equal to the number of classes and a `sigmoid` activation, and use `binary_crossentropy` as the loss. Your targets should be k-hot encoded:

```
model = models.Sequential()  
model.add(layers.Dense(32, activation='relu', input_shape=(num_input_features,)))  
model.add(layers.Dense(32, activation='relu'))  
model.add(layers.Dense(num_classes, activation='sigmoid'))  
model.compile(optimizer='rmsprop', loss='binary_crossentropy')
```

- To perform *regression* toward a vector of continuous values, end your stack of layers with a Dense layer with a number of units equal to the number of values you're trying to predict (often a single one, such as the price of a house), and no activation. Several losses can be used for regression, most commonly `mean_squared_error` (MSE) and `mean_absolute_error` (MAE):

```
model = models.Sequential()  
model.add(layers.Dense(32, activation='relu', input_shape=(num_input_features,)))  
model.add(layers.Dense(32, activation='relu'))  
model.add(layers.Dense(num_values))  
model.compile(optimizer='rmsprop', loss='mse')
```

Selecting an architecture: conv networks

- *Convnets*, or *convolutional networks*, consist of stacks of convolution and max-pooling layers. The pooling layers let you spatially downsample the data, which is required to keep feature maps to a reasonable size as the number of features grows, and to allow subsequent convolution layers to “see” a greater spatial extent of the inputs.
- Convnets are often ended with either a Flatten operation or a global pooling layer, turning spatial feature maps into vectors, followed by Dense layers to achieve classification or regression.
- Note that it’s highly likely that regular convolutions will soon be mostly (or completely) replaced by an equivalent but faster and representationally efficient alternative: the *depthwise separable convolution* (SeparableConv2D layer). This is true for 3D, 2D, and 1D inputs. The SeparableConv2D layer can be used as a drop-in replacement for Conv2D, resulting in a smaller, faster network that also performs better on its task. Here’s a typical image-classification network (categorical classification, in this case):

```
model = models.Sequential()  
model.add(layers.SeparableConv2D(32, 3, activation='relu', input_shape=(height, width, channels)))  
model.add(layers.SeparableConv2D(64, 3, activation='relu'))  
model.add(layers.MaxPooling2D(2))  
model.add(layers.SeparableConv2D(64, 3, activation='relu'))  
model.add(layers.SeparableConv2D(128, 3, activation='relu'))  
model.add(layers.MaxPooling2D(2))  
model.add(layers.SeparableConv2D(64, 3, activation='relu'))  
model.add(layers.SeparableConv2D(128, 3, activation='relu'))  
model.add(layers.GlobalAveragePooling2D())  
model.add(layers.Dense(32, activation='relu'))  
model.add(layers.Dense(num_classes, activation='softmax'))  
model.compile(optimizer='rmsprop', loss='categorical_crossentropy')
```

Visualizing intermediate convnet outputs (I)

- Deep NNs have been criticized for being a “black box” where the “knowledge” (the rules learned by the network) is hidden and not interpretable by humans. However, the representations learned by convnets are highly amenable to visualization, and today, a wide array of techniques have been developed for visualizing and interpreting these representations.
- **Visualizing the model's architecture.** Useful to know the layer structure of the model and the dimension of the input, output volumes, and the dimension of the filters. Use function:
 - `model.summary()` to get a text description, and
 - `plot_model` for a figure.
- **Visualizing intermediate convnet outputs** (intermediate activations for a given input image). Useful for understanding how successive convnet layers transform their input, and for getting a first idea of the meaning of individual convnet filters.
- **Visualizing convnets filters.** Useful for understanding precisely what visual pattern or concept each filter in a convnet is receptive to.

Visualizing intermediate convnet outputs (II)

- **Visualizing inputs for Activation Maximization.** Generating and Visualizing the input that maximizes a filter response. That is, another easy way to inspect the filters learned by convnets is to display the visual pattern that each filter is meant to respond to.
 - This can be done automatically with `keras.vis` library:
 - `visualize_activation`
 - Can be used for conv filter visualization, but also for Categorical Output Dense layer visualization, or Regression Output Dense layer visualization:
 - `input_img += gradient(activation, model.input)`
- **Visualizing heatmaps of class activation for a given input image.** Useful for understanding which parts of an image were identified as belonging to a given class, thus allowing you to localize objects in images.
 - `compute_gradient(output_category, last_conv_layer.output)`
 - This can also be done automatically with `keras.vis`, which provides two functions:
 - `visualize_saliency`
 - `visualize_cam`

Visualizing intermediate convnet outputs (III)

